Part 3: Appendices — Supporting Documentation

Cross-Modal Consistency Is Not a Cross-Generator Invariant:
An Attributed Negative-Result Study of
Lightweight Frozen-Backbone Deepfake Detection

Monash University — FIT5128/FIT4444/FIT4448 Thesis Assignment
Student: Chris Feng (36310085) Supervisor: Asad Malik 2026

This part is not assessed; it is provided for examiners per Assignment 3 Annex B. It documents, for audit purposes only, (A) the submitted code archive and three core code excerpts, (B) the datasets and file formats used, (C) the software environment and reproduction commands, (D) the index that links every reported number to its raw evaluation logs, and (E) the relationship of this material to the assessed paper in Part 2. The Ethics and Data Privacy Statement for this project is given in Part 2 and is not repeated here.

Contents

1 Appendix A: Code Organisation and Inventory **2**

2 Appendix B: Datasets and File Formats **5**

3 Appendix C: Experimental Environment and Reproduction **7**

4 Appendix D: Evidence-Chain Index **8**

5 Appendix E: Relationship to the Assessed Material **10**

1 Appendix A: Code Organisation and Inventory

The submitted code archive `final_code_20261019/` is a curated copy of the original working directory, re-organised into numbered stages of the experiment pipeline (the original `code_cloud/` working directory was left untouched; scripts use flat imports, so they are run either from the original directory or after copying the sub-folder contents back into one flat directory). Directory tree (text form, file counts in brackets, from the archive `README.md`):

```
final_code_20261019/
|-- 01_data_prep/       [20]  dataset preprocessing: prep_deepspeak, prep_voxceleb2,
|                             prep_fakeavceleb, prep_eval2024, prep_hifi, ... (13 scripts),
|                             extract_features.py -> per-clip .pt feature caches
|-- 02_core/            [10]  models.py (Sync-X / LCMA / LACA / LTCA), dataset.py
|                             (cache reader), losses.py (two-stage losses + GenD),
|                             config.py, common.py, utils.py, logging_utils.py, ...
|-- 03_training/        [3]   train_stage1.py (VoxCeleb2 self-supervised),
|                             train_stage2.py (DeepSpeak supervised, align/GenD switches),
|                             hyperparameter_search.py
|-- 04_evaluation/      [6]   evaluate.py (cross-domain zero-shot, modality masking,
|                             --fta_no_sync ablation) -> jsonl, dump_embeddings.py,
|                             dump_features.py, dump_interpret.py, b6_* shortcut audits
|-- 05_diagnostics/     [11]  diag_shortcut / physics / freq / sync_t / audio_probe /
|                             multigen_prior / carrier_probe / crossdomain / real_prior /
|                             train_cache / clip_duration  (evidence-chain probes)
|-- 06_figures/         [6+]  make_figs.py, make_extra_figs.py, plot_figures.py,
|                             plot_style.py, figures/, FIGURE_PIPELINE.md
|-- 07_cloud_jobs/      [17]  cloud_run_all / eval_only / settle_*.sh, cloud_abl_gend.sh
|                             (GenD ablation), Dockerfile, requirements.txt, runbook
|-- 08_docs/            [5]   original README, script guide, hyperparameter guide,
|                             ablation-C notes, dataset upload flow
|-- 99_dev_tools/       [27]  smoke tests, _verify_*, _smoke_*, bench (development aids)
|-- 99_archive/         [10]  historical module zip snapshots (experiment history)
|-- weights/                  face_landmarker.task (MediaPipe landmarker model)
`-- README.md                 archive overview and key entry points
```

Update package (24 October 2026). The module-ablation re-runs and the expanded carrier probe reported in Part 2 were executed with a six-file package, `syncx_final_code_20261024.zip`, extracted over the working directory `code_cloud/`: `models.py` and `train_stage2.py` (both patched, see below) plus four new scripts — `train_laca.py` (sets `cfg.laca_mode` for the LACA-branch ablations), `run_module_ablation.py` (train/eval/summary driver for the four variants), `report_seed_stats.py` (mean±std from existing logs) and `run_carrier_probe_expanded.py` (carrier probe with raised per-class caps). Two defects were fixed in place, without adding parameters or changing the checkpoint structure: (i) `models.py` fed a 512-D fused vector to the 256-D audio classification head in the `-disable_lcma` branch, raising a shape error, so that branch now projects the fused vector to `d_model` before classification while retaining the 512-D vector for the correlation loss; (ii) `train_stage2.py` dereferenced the shared-space alignment term when `laca_mode=biomech`, where that term is undefined, so the term is now skipped (counted as zero). Neither change alters any reported number for the principal configuration.

A.1 Core excerpt 1: the `LTCA` module (`02_core/models.py`)

Source: `final_code_20261019/02_core/models.py`, class defined at line 313 (docstring omitted), constructor/helper/forward reproduced verbatim from lines 332–368. The same class is present in the submitted `code_cloud/models.py`; the 24 October patch touches only the `-disable_lcma` branch further down the file, so the excerpt and its line numbers are unaffected.

```
class LTCA(nn.Module):
```

```
    def __init__(self, cfg):
        super().__init__()
        self.cfg = cfg
        d = cfg.d_model; H = cfg.fta_hidden
        self.aud_conv = self._conv_stack(d, cfg.fta_dilation)
        self.vis_conv = self._conv_stack(d, cfg.fta_dilation)
```

```
        self.pool_proj = nn.Linear(2 * d, 2 * d)
        self.ff = nn.Sequential(nn.Linear(2 * d, H), nn.GELU(), nn.Linear(H, 2 * d))
        self.cross_attn = nn.MultiheadAttention(d, cfg.n_heads, cfg.dropout, batch_first=True)
        self.sync_proj = nn.Linear(2 * d, 2 * d)

    @staticmethod
    def _conv_stack(d, dilations):
        layers = []
        for k in dilations:
            layers.append(nn.Conv1d(d, d, kernel_size=3, padding=k, dilation=k))
            layers.append(nn.GELU())
        return nn.Sequential(*layers)

    def forward(self, a_seq, v_seq, no_sync=False):
        aT = self.aud_conv(a_seq.transpose(1, 2)).transpose(1, 2)
        vT = self.vis_conv(v_seq.transpose(1, 2)).transpose(1, 2)
        a_p, v_p = aT.mean(1), vT.mean(1)
        h_long = self.ff(self.pool_proj(torch.cat([a_p, v_p], dim=-1)))
        if no_sync:
            return h_long
        v_sync, _ = self.cross_attn(v_seq, a_seq, a_seq)
        sync_t = F.cosine_similarity(a_seq, v_seq, dim=-1).mean(dim=1)
        h_av = v_sync.mean(1)
        h_sync = self.sync_proj(torch.cat([h_av, a_p], dim=-1)) * (0.5 + 0.5 * sync_t.unsqueeze
    → (-1))
        return h_long + h_sync
```

Typesetting note: source comments in the archive are written in Chinese; under pdfTEX characters outside the T1 font encoding are silently omitted from verbatim listings, so some inline comments appear truncated or blank. The executable code tokens are reproduced exactly as in the source files.

A.2 Core excerpt 2: the `-fta_no_sync` evaluation ablation (`04_evaluation/evaluate.py`)

Source: `final_code_20261019/04_evaluation/evaluate.py`, lines 188–191 (argument declaration), line 208 (switch applied to the loaded checkpoint's config), and lines 106–108 (variant labelling of the output record).

```
    ap.add_argument("--fta_no_sync", action="store_true",
                    help="w/o FTA-sync: LTCA keeps only long backbone h_long, skips subsystem B
    → sync (inference only, no weight change)")
```

```
    model.cfg.fta_no_sync = bool(getattr(args, "fta_no_sync", False))
```

```
    if getattr(args, "fta_no_sync", False):
        v = v + "_fta_no_sync"
    return v
```

When `-fta_no_sync` is passed, `LTCA.forward` takes the `no_sync` branch shown in A.1 and returns only the long-term backbone `h_long` — the single-variable "remove the audio–visual sync scalar" ablation; no weights are retrained, and the run is recorded under the `_fta_no_sync` variant suffix in its jsonl output.

A.3 Core excerpt 3: reading the `.pt` feature cache (`02_core/dataset.py`)

Source: `final_code_20261019/02_core/dataset.py`, `__getitem__` method, lines 153–185.

```
    def __getitem__(self, idx):
        f = self.files[idx]
        d = torch.load(f, map_location="cpu", weights_only=True)
        audio_seq = d["audio_seq"].float()
        v_clip = d["v_clip"].float()
        v_lip = d["v_lip"].float()
```

```
        label = int(d.get("label", 0))
        identity = str(d.get("identity", ""))
        source_id = str(d.get("source_id", ""))
        if self.temporal_jitter:
            audio_seq, v_clip, v_lip = self._jitter_temporal(audio_seq, v_clip, v_lip)
        audio_seq = _fix_len(audio_seq, AUDIO_LEN, dim=0)
        v_clip = _fix_len(v_clip, VIS_LEN, dim=0)
        v_lip = _fix_len(v_lip, VIS_LEN, dim=0)
        return {
            "audio_seq": audio_seq,
            "v_clip": v_clip,
            "v_lip": v_lip,
            "label": label,
            "identity": identity,
            "source_id": source_id,
            "path": f,
        }
```

2 Appendix B: Datasets and File Formats

B.1 Datasets used (roles, scale, licence and source)

Table 1: Dataset roles, format/scale highlights, licences and sources. Raw third-party videos are never redistributed with the submission; only derived feature caches and evaluation logs are retained.

Dataset	Role	Format / scale highlights	Licence & source
DeepSpeak v1.1	Primary training corpus (Stage-2 supervised)	Face-swap, lip-sync, avatar and voice-cloned audio forgeries; identity-disjoint 70/10/20 train/val/test split; 10,550 training and 2,913 test clips cached	Barrington et al., 2024 (public release); used under its release terms
VoxCeleb2	Stage-1 self-supervised pre-training, authentic clips only	Local subset: 500 speakers, 89,135 real interview clips (no forgeries)	Nagrani et al., VGG, University of Oxford; research-use terms
FakeAVCeleb	Cross-domain evaluation + B6 silence-shortcut probe	178 real / 19,547 fake (extreme imbalance); 19,725 usable clips once the 17,849 single-clip B6 items are excluded	Khalid et al., 2022 (v1.2 release); research release terms
Deepfake-Eval-2024	Cross-domain evaluation (in-the-wild, primary)	`video-data/*.mp4` + `video-metadata` CSV; 815 videos downloaded, 728 clips preprocessed (393 real / 335 fake; 87 extraction failures)	Chandra et al., 2025; HuggingFace `nuriachandra/Deepfake-Eval-2024` (gated), CC-BY-SA-4.0, **evaluation only, no training**
HiFi-AVDF	Cross-domain evaluation (second-generation generators)	1,905 real + 1,905 fake released; 3,787 usable clips after preprocessing (1,891 real / 1,896 fake); six commercial generators: Veo 3.1, Kling 2.5, Seedance 1.0, PixVerse V5, WAN 2.5, Sora 2	HuggingFace `JielunPeng/HiFi-AVDF`; used under its release terms

B.2 Per-clip feature cache (`.pt`) keys

Key	Type / shape	Meaning
`audio_seq`	float tensor $(T_a, 768)$	wav2vec2 audio content embeddings
`v_clip`	float tensor $(T_v, 512)$	CLIP frame embeddings (whole-face crop)
`v_lip`	float tensor $(T_v, 192)$	lip landmarks (64×3 coordinates)
`label`	int	class label (0 = real, 1 = fake)
`identity`	str	subject identity (split construction / leakage control)
`source_id`	str	source-clip id (same-source pairing, "Rec C"; empty if absent)
`path`	str	path of the source clip (carried through the dataset item)

Each cache file holds one clip of ≈ 4 s: the loader pads/truncates the three modalities to fixed lengths, and the model operates on content embeddings with time steps $T = 50$ (≈ 4 s window) in LCMA/LTCA.

B.3 Evaluation log (`jsonl`) fields

Each line of `result/data/persist/logs/<exp_id>/eval/<dataset>__<variant>.jsonl` is one completed evaluation record:

Field	Type	Meaning
exp_id	str	run id encoding config, seed and tag, e.g. syncx_bs512_lam0.3_lacaboth_seed0_seed0
dataset	str	evaluation set (DFE2024, FakeAVCeleb, HiFiAVDF, ...)
split	str	split evaluated (e.g. test)
method	str	detector family (e.g. syncx)
variant	str	condition label (full, audio-only, video-only, wo_lcma, wo_dwf, wo_xs, fta_no_sync, b6_trimmed, ...)
seed	int	random seed of the run
global_step	int or null	training step of the evaluated checkpoint
metrics	object	AUC, AP, EER, ACC, F1
fpr, tpr	arrays	full ROC curve points
group	str or null	per-category / per-group bucket (if requested)
timestamp	str	ISO-8601 local timestamp of the evaluation

3 Appendix C: Experimental Environment and Reproduction

C.1 Environment

- Hardware: a single cloud GPU, **NVIDIA RTX 4090D, 24 GB**; all final experiments reported in Part 2 were run on this card.
- Framework: **PyTorch 2.8.0+cu128** with the matching **torchvision 0.23.0+cu128** (torch and torchvision must be installed as a paired combination). Evidence and dependency-alignment history: `result/v1/avhubert_cloud_pitfalls.md` ("pitfall 9"), together with the pinned versions recorded there (numpy 1.23.5, opencv 4.5.4, scipy 1.11.4) and the cloud interpreter notes in the same document.
- `07_cloud_jobs/requirements.txt` is an **early cloud snapshot** (torch 2.4.1 + cu121) and is provided for reference only; reproducing the reported numbers requires the cu128 pairing above, not the snapshot versions.
- Orchestration: `07_cloud_jobs/` (Dockerfile, `cloud_run_all.sh`, `cloud_eval_only.sh`, `cloud_abl_gend.sh`); system-level requirement: `ffmpeg` for video/audio decoding during preprocessing.

C.2 Training and evaluation commands

```
# Stage 1: self-supervised pre-training on authentic VoxCeleb2 clips
python train_stage1.py --real_cache <voxceleb2_real_cache> \
    --val_cache <voxceleb2_val_cache> --out_ckpt ckpt/stage1.pt

# Stage 2: supervised training on the DeepSpeak v1.1 caches
#   (loads the Stage-1 weights via --stage1_ckpt; required flags shown first)
python train_stage2.py --train_cache <deepspeak_train_cache> \
    --val_cache <deepspeak_val_cache> --stage1_ckpt ckpt/stage1.pt

# Zero-shot cross-domain evaluation -> jsonl record per (dataset, variant)
python evaluate.py --ckpt <checkpoint> --eval_cache <eval_cache>

# Single-variable ablations (examples; each writes its own variant suffix)
python evaluate.py --ckpt <checkpoint> --eval_cache <eval_cache> --mask audio
python evaluate.py --ckpt <checkpoint> --eval_cache <eval_cache> --fta_no_sync

# Module ablation (update package): four variants, one seed each
python run_module_ablation.py --stage all --seeds 0     # train + eval + summary
python run_module_ablation.py --stage summary           # re-summarise existing logs
python train_laca.py --laca_mode corr                   # or: biomech

# Expanded carrier probe (per-class caps raised fivefold; same verdict)
python run_carrier_probe_expanded.py --max_per_real 3000 --max_per_fake 3000
```

The training entry points above are `03_training/train_stage1.py` and `03_training/train_stage2.py`; evaluation is `04_evaluation/evaluate.py`. Argument names are quoted from the scripts' own `argparse` definitions.

C.3 Randomness

Main results use **seed 0** (the scripts' default `-seed 0`). The GenD component ablation (evidence chain §17) was run with **three seeds** (0, 1, 2) per variant and is reported as mean±std across those seeds; per-seed values are listed in §17.2 of the evidence chain and are each backed by the corresponding jsonl records (Appendix D).

4 Appendix D: Evidence-Chain Index

Every quantitative claim in Part 2 is traceable through two artefacts shipped with this submission:

1. `result/v1/evidence_chain_crossdomain.md` — the engineering evidence chain, sections §0 to §18 (including sub-sections §5.1–§5.2, §6B, §8B–§8H, §11.1–§11.4, §12.1–§12.3, §13.1–§13.6, §13b, §17.1–§17.4, §18.1–§18.6). It narrates, in chronological order, each falsification step of the cross-domain failure analysis: phenomenon (§0–§1), data/identity/generator exclusions (§2), modality masking (§3–§4), training-domain shortcut scan (§5), physical-proxy and carrier probes (§6–§6B), root-cause integration and training-side ablations (§7–§8H), interpretation rules (§10–§12), alignment/HAVIC/multi-source attempts (§13–§15), feature-carrier probe (§16), the GenD component ablation (§17) and the adapter module ablation plus the expanded carrier probe (§18).

2. `result/data/persist/logs/<exp_id>/eval/*.jsonl` — the raw evaluation records (schema in B.3) that back the numbers quoted in each section.

D.1 Evidence-chain section → backing artefacts (index §0–§17)

Table 2: Mapping from the sections of `result/v1/evidence_chain_crossdomain.md` to the scripts and raw logs that back them. Evaluation logs live under `result/data/persist/logs/<exp_id>/eval/`; training curves are each run's `stage2_train.jsonl`; diagnostic scripts are in `05_diagnostics/`.

Section	What it records	Principal backing artefacts
§0–§1	One-line conclusion; the cross-domain collapse phenomenon	seed-0 main config: `...lacaboth_seed0_seed0/eval/` `DFE2024__full.jsonl`, `FakeAVCeleb__full.jsonl`, `HiFiAVDF__full.jsonl`
§2	Data / identity / generator triple exclusion	split and metadata audits; the same baseline jsonl records
§3–§4	Modality masking; audio probe vs. fusion control	`...mask_audio_masked_seed0/eval/HiFiAVDF__audio-only.jso` `...mask_visual_masked_seed0/eval/HiFiAVDF__video-only.js` `diag_audio_probe.py`
§5 (§5.1–§5.2)	Training-domain shortcut scan; cross-generator transferability	`diag_shortcut.py` per-class cache scans
§6–§6B	Physical-proxy probes and frequency/temporal-coherence probes (Direction A falsified)	`diag_physics_probe.py`, `diag_freq_probe.py`
§7	Implementation bugs found during analysis	code comments / change markers in the archive
§8, §8B, §8C	Root-cause integration; frozen-encoder and visual-only-freeze negative results	the corresponding runs' `stage2_train.jsonl` + `eval/*.jsonl`
§8D–§8E	First LTCA measurement; HiFi sync-scalar single-variable check	`evaluate.py -fta_no_sync` runs; `diag_sync_t.py`
§8F	Feature survey (AV-HuBERT / consistency priors / ROGA)	literature survey; no experiment run

Table 2 continued

Section	What it records	Principal backing artefacts
§8G–§8H	Audio–visual time-window misalignment hypothesis, then its falsification	`...windowfix_hifi_seed0/eval/HiFiAVDF__full.jsonl`; `diag_sync_t.py`
§10–§12	Interpretation rules; stage assessment; honest-negative framing	narrative sections (no new runs)
§13–§13b	End-to-end alignment pilot (group B): "seesaw" negative transfer	`...alignprior_groupB_seed0/eval/*.jsonl` + its `stage2_train.jsonl`
§14	HAVIC three-layer consistency prior fails to reproduce	`...havic_vs_base_seed0/eval/*.jsonl`
§15	Multi-source prerequisite probe: cross-generator sign instability	`diag_multigen_prior.py`, `diag_real_prior.py`
§16	Feature-carrier probe (the trigger for the AV-HuBERT decision)	`diag_carrier_probe.py` → `Figures/fig_carrier_probe`
§17 (§17.1–§17.4)	GenD component ablation (Rec B / Rec C), 3 seeds each	`...full_gend_seed{0,1,2}/eval/*__full.jsonl`, `...abl_nob_seed{0,1,2}/...` (w/o Rec B), `...abl_noc_seed{0,1,2}/...` (w/o Rec C), plus component switches `wo_xs`, `wo_dwf`, `laca_biomech`, `laca_corr`
§18 (§18.1–§18.6)	Adapter module ablation (LCMA / DWF / each LACA branch) and the expanded carrier probe	`abl_wo_lcma_seed0/`, `abl_wo_dwf_seed0/`, `abl_laca_corr_seed0/`, `abl_laca_biomech_seed0/` (each `eval/*.jsonl` plus `stage2_train.jsonl`); scripts `run_module_ablation.py`, `train_laca.py`, `run_carrier_probe_expanded.py`; summary `result/v1/module_ablation_and_probe_results_20261024.md`

D.2 One-line query: number → jsonl

To locate which raw record contains a given reported number, scan all evaluation logs:

```
python -c "import glob,json,os; [print(os.path.basename(os.path.dirname(os.path.dirname(f))), json.loads(l)['
    → dataset'], json.loads(l)['metrics']['AUC']) for f in glob.glob(r'C:\...\GraduationThesis\result\data\
    → persist\logs\*\eval\*.jsonl') for l in open(f,encoding='utf-8') if '0.5796' in l]"
```

and, for one specific record:

```
python -c "import json; d=json.loads(open(r'C:\...\GraduationThesis\result\data\persist\logs\syncx_bs512_lam0.3
    → _lacaboth_seed0_seed0\eval\HiFiAVDF__full.jsonl',encoding='utf-8').readline()); print(d['dataset'], d['
    → metrics']['AUC'], d['timestamp'])"
```

Each printed line identifies the `exp_id` (hence the exact run configuration and seed), the dataset and the metric value, closing the loop from any number in Part 2 back to the raw log.

D.3 Principal configuration: three-seed replication

The principal configuration was trained with three seeds. Every per-seed cross-domain AUC is listed below together with the mean and standard deviation, so the single-seed values quoted in Part 2 can be audited against the run-to-run spread. The records are `syncx_bs512_lam0.3_lacaboth_seed0_seed0`, `..._seed1_seed1` and `..._seed2_seed2`, each under `<exp_id>/eval/`.

Seed	FakeAVCeleb	DFE-2024	HiFi-AVDF
0	0.6187	0.5350	0.4735
1	0.5816	0.5279	0.4873
2	0.6214	0.5007	0.4892
Mean ± std	0.6072 ± 0.0222	0.5212 ± 0.0181	0.4833 ± 0.0086

The three standard deviations are exactly the values quoted in Part 2, which is why every non-LCMA ablation delta that falls inside them is reported as unresolved rather than as a benefit or a harm.

5 Appendix E: Relationship to the Assessed Material

This appendix is supporting documentation only: it is **not assessed** and **not counted in the Part 2 word count** (Part 2 is 7,024 words, excluding references, strict count), and every number reported in Part 2 can be traced back through the directory indexed in Appendix D to its raw `jsonl` record. The project's ethics position is stated once, in the Ethics and Data Privacy Statement of Part 2.